Universidade Estadual de Santa Cruz (UESC)

Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória compartilhada com OpenMP.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Enzo Fernandes Sarmento, Jéssica Meira Costa

Disciplina: DEC107 — Processamento Paralelo

Data: 22/09/2026

1. Introdução

O processamento paralelo tem como objetivo utilizar simultaneamente múltiplos recursos computacionais para reduzir o tempo necessário para executar determinados tipos de aplicações. Na disciplina, um dos principais aspectos estudados é a identificação de problemas que podem ser divididos em tarefas independentes e a análise de como diferentes estratégias de paralelização influenciam o desempenho da aplicação.

Neste contexto, o objetivo desta etapa é implementar uma versão sequencial e uma versão paralela do algoritmo de geração do Conjunto de Mandelbrot utilizando OpenMP, avaliando o impacto do número de threads, das políticas de escalonamento e do tamanho do chunk sobre o tempo de execução, speedup, eficiência e balanceamento de carga.

O Conjunto de Mandelbrot é obtido a partir da análise do comportamento de uma sequência de números complexos. Para cada ponto (c) do plano complexo, considera-se inicialmente $z(0) = 0$ e aplica-se repetidamente a expressão: $z(n+1) = z(n)^2 + c$.

O método de *escape-time* determina quantas iterações são necessárias para que o módulo de (z) ultrapasse um determinado raio de escape. Neste projeto, foi utilizado o critério: $|z|^2 > 4$ que é equivalente a ($|z|>2$), mas evita a necessidade de calcular uma raiz quadrada a cada iteração. O número de iterações realizadas, limitado pelo valor de MAX_ITER, corresponde ao valor armazenado para cada pixel da imagem.

A geração da imagem de Mandelbrot apresenta uma característica, na qual embora o cálculo de cada pixel possa ser realizado de forma independente, o custo computacional associado a cada pixel não é necessariamente igual. Pontos localizados fora do conjunto normalmente ultrapassam rapidamente o raio de escape, exigindo poucas iterações. Por outro lado, pontos pertencentes ao conjunto ou próximos à sua fronteira podem exigir muitas iterações, chegando ao limite máximo estabelecido.

Dessa forma, o custo das tarefas torna-se heterogêneo. Se as iterações do algoritmo forem distribuídas de maneira inadequada entre as threads, algumas podem terminar seu trabalho rapidamente enquanto outras continuam executando cálculos mais custosos. Esse fenômeno caracteriza o desbalanceamento de carga e constitui um dos principais aspectos analisados nesta etapa.

Para realizar a paralelização foi utilizada a API OpenMP, que fornece mecanismos para criação e gerenciamento de threads em programas escritos em C/C++. Entre os recursos utilizados estão as diretivas de paralelização de laços e as políticas de escalonamento static, dynamic e guided. Essas políticas determinam como as iterações do laço são distribuídas entre as threads.

Na static, as iterações são distribuídas previamente entre as threads. Essa estratégia possui baixa sobrecarga de gerenciamento, mas pode apresentar problemas quando as iterações possuem custos diferentes. Na política dynamic, as threads recebem novos blocos de iterações à medida que terminam os blocos anteriores, permitindo uma adaptação maior ao custo variável das tarefas. Já a guided começa distribuindo blocos maiores e reduz progressivamente o tamanho dos blocos, buscando combinar menor sobrecarga com melhor balanceamento.

A análise de desempenho é importante porque o simples aumento do número de threads não garante uma redução proporcional no tempo de execução. O desempenho pode ser afetado por fatores como desbalanceamento de carga, sincronização, criação

e gerenciamento das threads, acesso à memória e características do próprio processador. Por isso, além do tempo de execução, são analisadas métricas como Speedup, Eficiência e Fator de Balanceamento de Carga.

Assim, esta primeira etapa busca não apenas desenvolver uma versão paralela do benchmark, mas também compreender como diferentes estratégias de distribuição do trabalho afetam o comportamento de uma aplicação com custo computacional irregular em um ambiente de memória compartilhada.

2. Metodologia

2.1 Organização da implementação

A implementação foi organizada em um arquivo de cabeçalho e três arquivos-fonte principais:

- **mandelbrot.h:** definição das constantes, protótipos das funções e inclusão das bibliotecas utilizadas;
- **mandelbrot.c:** implementação das funções responsáveis pelo cálculo do Mandelbrot, configuração da região analisada e gravação dos resultados;
- **main.c:** implementação da execução sequencial;
- **main_openmp.c:** implementação da execução paralela com OpenMP.

Essa organização separa a lógica de cálculo do fractal das estratégias de execução utilizadas nos dois programas.

2.2 Configuração dos experimentos

Foram utilizados dois cenários.

Input padrão

No caso padrão foram utilizados:

- resolução: 4096 x 4096 pixels;
- `MAX_ITER=1000`;
- $Re_{in}[-2.0, 1.0]$;
- $Im_{in}[-1.5, 1.5]$.

A região é simétrica em relação ao eixo real, permitindo a utilização da otimização de simetria implementada no programa.

Vale dos Cavalos-Marinhos

Como teste de estresse, foi utilizada uma região do chamado Vale dos Cavalos-Marinhos, com:

- centro real: (-0.743643887);
- centro imaginário: (0.131825904);
- largura no eixo real: (3.0×10^{-3});
- resolução: (4096 x 4096);
- (`MAX_ITER=5000`).

Como a resolução utilizada é quadrada e o código utiliza a mesma largura espacial para os dois eixos, a extensão no eixo imaginário também corresponde 3.0×10^{-3}. Diferentemente do caso padrão, o cenário do Vale dos Cavalos-Marinhos não utiliza a otimização de simetria axial. Isso ocorre porque a região analisada está centrada em uma coordenada imaginária diferente de zero, fazendo com que o domínio selecionado não seja simétrico em relação ao eixo real.

2.3 Implementação sequencial de referência

A implementação sequencial foi desenvolvida como referência para a avaliação de desempenho e para a verificação da corretude da versão paralela. A lógica de cálculo do conjunto de Mandelbrot é compartilhada com a versão OpenMP por meio das funções definidas no arquivo mandelbrot.c.

A função `configure_view()` define os limites da região do plano complexo, os passos utilizados para mapear os pixels para coordenadas complexas e o número máximo de iterações. Para o caso padrão, são utilizados os limites

$$Re \in [-2.0,\ 1.0]$$

e

$$Im \in [-1.5,\ 1.5]$$

com resolução de 4096 × 4096 pixels e `MAX_ITER = 1000`.

Para cada pixel, a função `compute_pixel_at()` realiza o mapeamento entre as coordenadas inteiras do pixel e o plano complexo:

$$c_{re} = Re_{min} + px \cdot Re_{step}$$

$$c_{im} = Im_{min} + py \cdot Im_{step}.$$

Em seguida, a função `compute_pixel()` executa a recorrência do conjunto de Mandelbrot. Antes do processo iterativo, são realizadas duas verificações geométricas que identificam pontos pertencentes ao cardioide principal e ao bulbo principal. Quando uma dessas condições é satisfeita, a função retorna diretamente `max_iter`, evitando a execução das iterações para esses pontos.

Para os demais pontos, a sequência é calculada utilizando as componentes real e imaginária separadamente. A condição de escape é dada por

$$z_{re}^2 + z_{im}^2 > 4,$$

ou pela chegada ao limite `max_iter`.

Exploração da simetria

No caso padrão, a região do plano complexo é simétrica em relação ao eixo real. A implementação verifica essa propriedade automaticamente e, quando ela está presente, calcula somente aproximadamente metade das linhas da imagem.

Para uma imagem de altura H, é utilizado:

$$limit_y = \frac{H+1}{2}.$$

Após calcular uma linha `py`, o resultado é copiado para a linha correspondente ao outro lado do eixo real:

$$mirror_py = H - 1 - py.$$

Dessa forma, para o caso padrão de 4096 × 4096 pixels, apenas 2048 linhas são efetivamente calculadas, enquanto as demais são obtidas por espelhamento.

Essa otimização está presente tanto na implementação sequencial quanto na implementação paralela, garantindo que ambas utilizem a mesma estratégia de cálculo.

2.4 Implementação paralela com OpenMP

Na versão paralela, o processamento das linhas da imagem foi dividido entre diferentes threads usando OpenMP.

A região paralela foi criada com a diretiva:

```
#pragma omp parallel
```

Dentro dela, o laço que percorre as linhas usa:

```
#pragma omp for schedule(runtime) nowait
```

O omp for é responsável por dividir as iterações do laço entre as threads. Foi usado `schedule(runtime)` porque o tipo de escalonamento e o tamanho do chunk são escolhidos durante a execução. Isso facilitou a realização dos testes com diferentes configurações sem precisar alterar o código.

O número de threads é definido pelo programa com `omp_set_num_threads()`.

A opção nowait foi usada para evitar a barreira automática logo após o omp for. Assim, as threads não precisam esperar nesse ponto antes de continuar. A sincronização acontece no final da região parallel.

Não foi utilizado collapse, pois o programa divide diretamente o laço das linhas entre as threads. Depois que uma thread recebe uma linha, ela percorre normalmente todas as colunas daquela linha.

O código usa a simetria da imagem em relação ao eixo real. Quando essa condição é encontrada, parte das linhas é calculada e os resultados são copiados para as linhas correspondentes do outro lado. Essa otimização também é usada na versão sequencial, mantendo o mesmo cálculo nas duas versões.

2.5 Políticas de escalonamento e tamanho do chunk

Foram testadas três políticas de escalonamento do OpenMP:

- `static`;
- `dynamic`;
- `guided`.

Também foram utilizados três tamanhos de chunk:

- 1;
- 16;
- 64.

A política static distribui as iterações entre as threads antes do início do processamento. No dynamic, as threads vão recebendo novos blocos conforme terminam os anteriores. No guided, os blocos começam maiores e vão diminuindo durante a execução.

Essas opções foram testadas porque o custo para calcular cada pixel do conjunto de Mandelbrot pode ser diferente. Alguns pixels escapam rapidamente, enquanto outros precisam de muitas iterações. Por isso, a forma como as linhas são distribuídas pode influenciar o tempo de execução e o balanceamento entre as threads.

No caso padrão, foram testadas de 2 até 12 threads, juntamente com as três políticas e os três tamanhos de chunk.

Também foi realizado um teste na região chamada Seahorse Valley. Nesse caso, foi utilizado o centro (-0.743643887, 0.131825904), uma largura de 3.0e-3 e `MAX_ITER = 5000`. Esse teste aumenta o custo do cálculo e permite observar o comportamento do programa em uma região mais pesada do conjunto de Mandelbrot.

2.6 Hardware e ambiente de execução

Os testes foram realizados em um computador com o seguinte ambiente:

- Sistema operacional: Ubuntu 24.04.1;
- Kernel: Linux 6.14.0-37-generic;
- Arquitetura: x86_64;

- Processador: AMD Ryzen 5 5600H with Radeon Graphics;
- Núcleos físicos: 6;
- Threads lógicas: 12;
- Threads por núcleo: 2;
- Frequência máxima: 4,28 GHz;
- Cache L3: 16 MiB;
- Cache L2: 3 MiB;
- Cache L1d: 192 KiB;
- Cache L1i: 192 KiB;
- NUMA: 1 nó;
- Compilador: GCC 13.3.0.

O programa foi compilado com as opções `-O3`, `-march=native`, `-ffast-math` e `-fopenmp`.

Como o processador possui seis núcleos físicos e 12 threads lógicas, as configurações com mais de seis threads passam a utilizar o recurso de SMT do processador, embora o mapeamento efetivo das threads seja gerenciado pelo sistema operacional e pelo runtime OpenMP.

2.7 Medição do tempo

O tempo usado para comparar as versões foi o tempo gasto no cálculo dos pixels. A gravação dos arquivos .bin e .ppm não foi incluída no tempo utilizado para avaliar o desempenho computacional.

Na versão OpenMP, o tempo foi medido usando `omp_get_wtime()`. O cronômetro começa antes da região paralela e termina depois que as threads terminam o processamento. O tempo de gravação dos arquivos é medido separadamente.

Na versão sequencial, o tempo também foi medido somente durante o cálculo dos pixels, sem incluir a gravação dos arquivos.

Para cada configuração do benchmark, foram realizadas cinco execuções independentes. A média dos tempos foi utilizada como valor de referência para as comparações, e o desvio padrão foi utilizado para avaliar a variação entre as execuções.

No benchmark do caso padrão, foram testadas diferentes quantidades de threads, além das três políticas de escalonamento (static, dynamic e guided) e dos três tamanhos de chunk utilizados: 1, 16 e 64.

2.8 Métricas utilizadas

Foram utilizadas quatro métricas principais para analisar os resultados: tempo de execução, speedup, eficiência e fator de balanceamento.

Tempo de execução

O tempo da versão sequencial é representado por T1. O tempo da versão paralela, utilizando p threads, é representado por Tp.

Speedup

O speedup mostra quanto a versão paralela melhorou em relação à versão sequencial. O cálculo utilizado foi:

Speedup = T1 / Tp

Por exemplo, um speedup igual a 4 significa que, naquela configuração, a versão paralela foi aproximadamente 4 vezes mais rápida que a versão sequencial.

Eficiência

A eficiência mostra a relação entre o speedup e a quantidade de threads utilizadas:

Eficiência = Speedup / p

Também pode ser apresentada em porcentagem:

Eficiência (%) = (Speedup / p) × 100

Fator de Balanceamento de Carga

O programa registra o tempo de processamento de cada thread. A partir desses valores, é calculado o fator de balanceamento:

Fator de Balanceamento = T_médio / T_máximo

Nesse cálculo, T_médio é o tempo médio das threads e T_máximo é o maior tempo registrado entre elas.

Quanto mais próximo de 1 for o resultado, mais próximos são os tempos das threads. Valores menores indicam uma diferença maior entre esses tempos.

Além disso, o programa registra os tempos mínimo, máximo e médio das threads. Esses valores ajudam a observar como o trabalho foi distribuído entre as threads durante cada execução.

3. Resultados

3.1 Verificação da corretude

A primeira etapa da avaliação consistiu em verificar se a paralelização modificou o resultado produzido pelo algoritmo. Como o objetivo da versão OpenMP é reduzir o tempo de execução mantendo exatamente o mesmo cálculo realizado pela versão sequencial, a corretude foi verificada comparando-se os arquivos binários produzidos pelas duas implementações.

Para cada cenário, foram calculados os hashes SHA-256 dos arquivos de saída e realizada a comparação bit a bit. Os resultados estão apresentados na Tabela 1.

Tabela 1 — Verificação de corretude entre as versões sequencial e OpenMP

Cenário	**SHA-256**	**Divergências**
Padrão	hash serial = hash OpenMP	0
Seahorse	hash serial = hash OpenMP	0

No input padrão, os arquivos produzidos pelas duas implementações apresentaram exatamente o mesmo hash SHA-256, indicando que não houve qualquer alteração no conteúdo da matriz de iterações. A comparação também identificou zero pixels divergentes. O mesmo comportamento foi observado no teste do Vale dos

Cavalos-Marinhos, mesmo com o aumento de `MAX_ITER` de 1000 para 5000. Os valores registrados na tabela de corretude confirmam essa identidade exata.

Esse resultado é importante porque demonstra que as alterações introduzidas pela paralelização não modificaram a lógica do cálculo. As threads trabalham sobre diferentes linhas da imagem, mas cada pixel continua sendo calculado a partir das mesmas coordenadas complexas e das mesmas condições de escape.

Assim, os resultados de desempenho apresentados nas seções seguintes podem ser comparados diretamente com a implementação sequencial de referência.

3.2 Desempenho no caso padrão

Para o input padrão, foram utilizadas cinco repetições independentes para cada configuração, variando-se o número de threads de 2 a 12, as políticas static, dynamic e guided e os tamanhos de chunk 1, 16 e 64.

O tempo médio da versão sequencial utilizado como referência foi de **0,420650s**, considerando as cinco execuções realizadas.

A Tabela 2 apresenta, para cada quantidade de threads, a configuração que apresentou o menor tempo médio entre as nove combinações de política e chunk.

Tabela 2 – Melhor configuração média para cada número de threads no input padrão

Threads	Escalonamento	Chunk	Tp médio (s)	Speedup	Eficiência
2	dynamic	1	0,213194	1,973	98,65%
3	static	1	0,143088	2,940	97,99%
4	static	1	0,108095	3,891	97,29%
5	static	1	0,087165	4,826	96,52%
6	dynamic	1	0,073515	5,722	95,37%
7	dynamic	1	0,065915	6,383	91,19%
8	dynamic	1	0,058814	7,153	89,42%
9	dynamic	1	0,054453	7,727	85,86%
10	dynamic	1	0,050268	8,373	83,73%

11	static	1	0,050852	8,453	76,85%
12	dynamic	1	0,046265	9,147	76,22%

Os resultados mostram uma redução consistente do tempo de execução com o aumento do número de threads. Com duas threads, o tempo médio foi reduzido para aproximadamente 0,2132 s, correspondendo a um speedup de 1,973×. Ao utilizar seis threads, o tempo chegou a 0,0735 s, com speedup de 5,722×.

A partir de sete threads, observa-se uma redução progressivamente menor do tempo em relação ao aumento do número de threads. Esse comportamento está associado ao fato de que o processador utilizado possui seis núcleos físicos e 12 threads lógicas. Dessa forma, a partir de sete threads, duas threads passam a compartilhar os recursos de pelo menos um núcleo físico.

Com 12 threads, foi obtido o menor tempo médio do caso padrão, de **0,0463 s**, correspondendo a um speedup de **9,147×** em relação à execução sequencial. A eficiência correspondente foi de **76,22%**.

A Figura 1 apresenta visualmente a redução do tempo de execução conforme o número de threads aumenta.

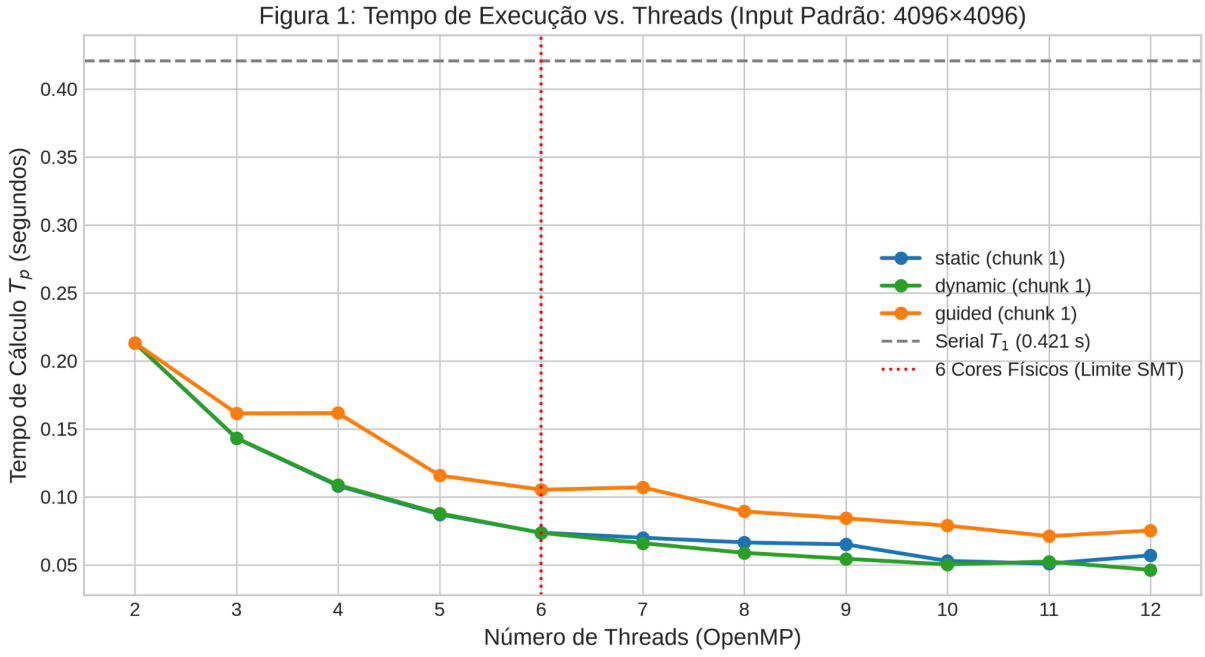


Figura 1: Tempo de execução em função do número de threads no input padrão

A curva mostra que o ganho é expressivo principalmente até a utilização dos seis núcleos físicos. Depois desse ponto, o tempo continua diminuindo, porém com ganhos progressivamente menores.

A Figura 2 apresenta o comportamento do speedup.

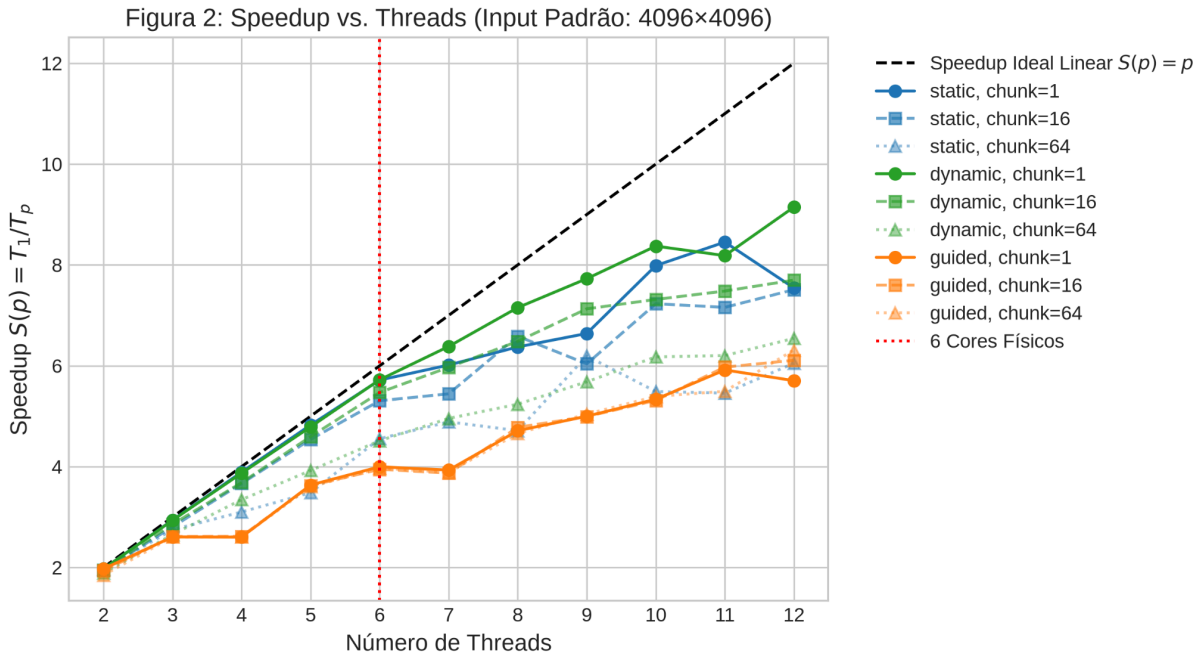


Figura 2: Speedup em função do número de threads no input padrão.

O speedup apresenta comportamento aproximadamente linear entre duas e seis threads. Entretanto, a curva se distancia progressivamente do speedup ideal à medida que mais threads são utilizadas. Essa diferença representa os custos associados ao gerenciamento das threads, ao escalonamento das iterações e ao compartilhamento dos recursos físicos do processador.

A eficiência correspondente é apresentada na Figura 3.

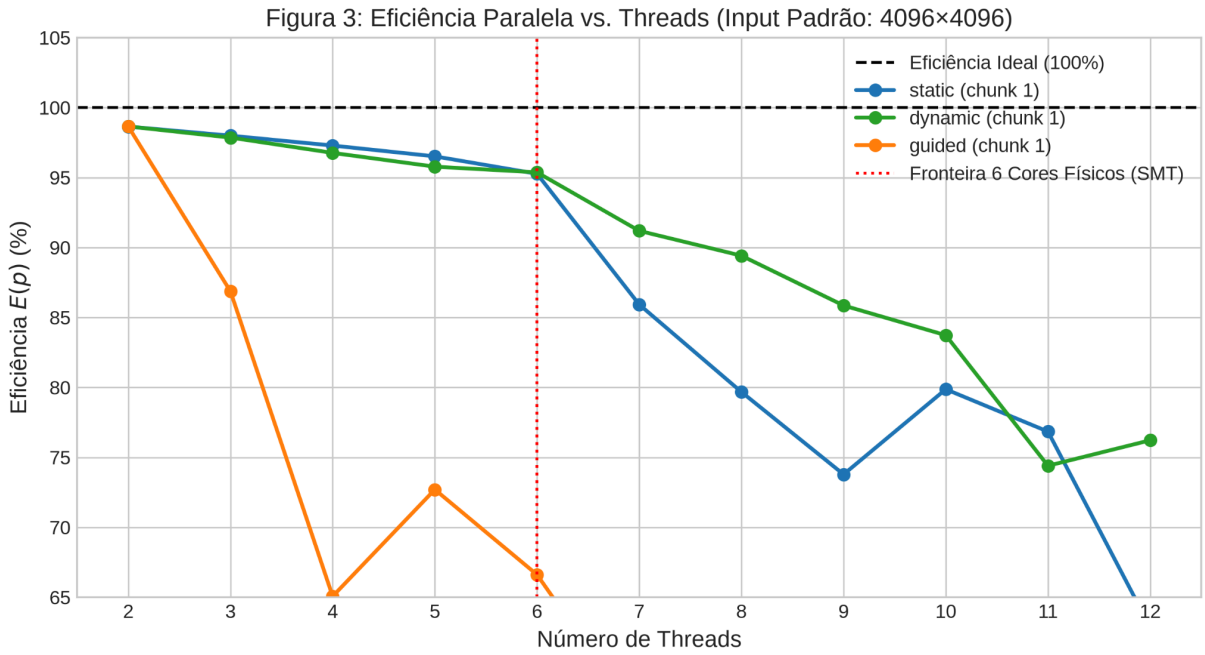


Figura 3: Eficiência em função de threads no input padrão.

A eficiência permanece superior a 95% até seis threads, indicando uma boa utilização dos núcleos físicos. A partir de sete threads, ocorre uma redução mais acentuada, chegando a 76,22% com 12 threads. Portanto, embora o maior número de threads continue proporcionando menor tempo absoluto, o ganho adicional por thread torna-se progressivamente menor.

3.3 Comparação entre Políticas de Escalonamento e Chunks (Caso Padrão)

Para investigar especificamente o efeito das políticas de escalonamento, foram comparadas as nove combinações de política e tamanho de chunk utilizando 12 threads. Essa configuração foi escolhida por representar a utilização máxima das threads disponíveis no processador.
A Tabela 3 apresenta os resultados obtidos.

Tabela 3 - Comparação dos escalonamentos com 12 threads no input padrão

Escalonamento	Chunk	Tp médio (s)	Speedup médio	Eficiência média	LBF médio
dynamic	1	0,046265	9,147	76,22%	0,9822
dynamic	16	0,055402	7,699	64,16%	0,8804

static	16	0,056774	7,509	62,58%	0,7941
static	1	0,056956	7,544	62,87%	0,8282
dynamic	64	0,064414	6,544	54,53%	0,7049
guided	64	0,066707	6,312	52,60%	0,6501
guided	16	0,069191	6,109	50,91%	0,6289
static	64	0,069975	6,050	50,42%	0,6142
guided	1	0,075219	5,705	47,54%	0,6132

Os resultados evidenciam uma relação importante entre o tamanho do chunk e o balanceamento de carga. A configuração dynamic, chunk=1 apresentou simultaneamente o menor tempo de execução, o maior speedup e o maior fator de balanceamento entre as configurações analisadas.

Nesse contexto, é importante observar que chunk=1 não corresponde a um único pixel. Como o laço paralelizado é o laço das linhas, cada iteração distribuída pelo OpenMP corresponde a uma linha completa da imagem. Portanto, com chunk=1, cada unidade de trabalho possui 4096 pixels.

Essa característica explica parte do bom desempenho observado. A unidade de trabalho continua sendo suficientemente grande para representar uma quantidade significativa de computação, mas suficientemente pequena para permitir que as threads recebam novas linhas à medida que terminam suas tarefas.

A Figura 4 apresenta a comparação entre as políticas e os diferentes tamanhos de chunk.

Figura 4: Speedup Médio por Política e Chunk Size no Input Padrão

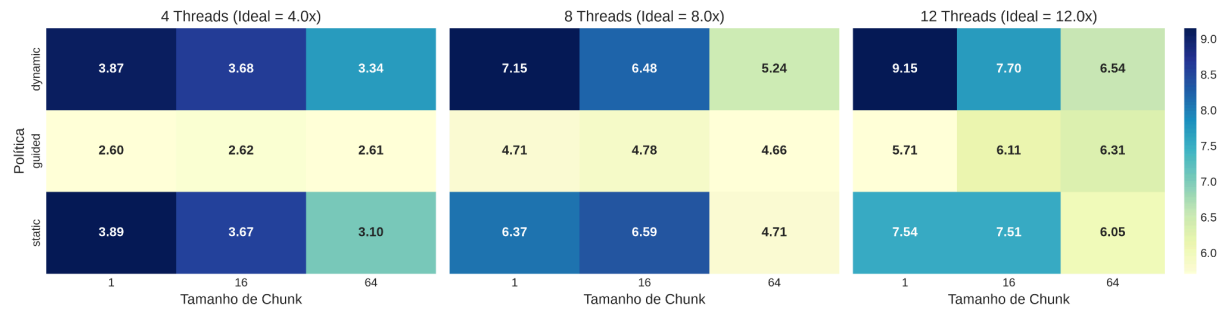


Figura 4: Efeito da política de escalonamento e do tamanho do chunk sobre o desempenho.

A diferença entre as configurações torna-se mais evidente conforme aumenta o número de threads. Para quatro threads, várias configurações apresentam comportamentos relativamente próximos. Com 12 threads, entretanto, as diferenças são maiores, principalmente nos chunks 64.

O resultado também mostra que aumentar o chunk não necessariamente melhora o desempenho. Embora chunks maiores reduzam a frequência de redistribuição das tarefas, eles também agrupam mais linhas em uma mesma atribuição. Como o custo das linhas não é uniforme, essa agregação pode prejudicar o balanceamento.

3.4. Teste de Estresse – Vale dos Cavalos-Marinhos

Para avaliar o comportamento da implementação em uma região com maior custo computacional, foi utilizado o Vale dos Cavalos-Marinhos, com centro em:

$$(-\ 0.743643887, 0.131825904)$$

e largura de 3.0×10^{-3}, utilizando `MAX_ITER=5000`.

Esse cenário aumenta consideravelmente o custo de processamento dos pixels, uma vez que uma parcela significativa deles necessita de muitas iterações antes de atingir o critério de escape.

A Tabela 4 apresenta as melhores configurações encontradas para cada quantidade de threads.

Tabela 4 - Melhor configuração média para cada número de threads no Vale dos Cavalos-Marinhos

Threads	Escalonamento	Chunk	Tp médio (s)	Speedup médio	Eficiência média	LBF médio
2	static	64	1,538036	1,982	99,10%	0,9990
3	static	1	1,031975	2,954	98,48%	0,9995
4	dynamic	1	0,774445	3,936	98,41%	0,9993
5	dynamic	1	0,625952	4,870	97,40%	0,9991

6	static	1	0,527793	5,776	96,27%	0,9949
7	dynamic	1	0,461427	6,607	94,38%	0,9988
8	dynamic	1	0,414315	7,360	91,99%	0,9985
9	dynamic	1	0,374637	8,140	90,45%	0,9983
10	dynamic	1	0,337661	9,028	90,28%	0,9982
11	dynamic	1	0,327285	9,353	85,03%	0,9955
12	dynamic	1	0,306488	9,972	83,10%	0,9902

A principal diferença em relação ao input padrão aparece no tempo absoluto de execução. Enquanto o melhor resultado com 12 threads no caso padrão foi de aproximadamente 0,0463 s, o mesmo número de threads no Vale dos Cavalos-Marinhos apresentou tempo de 0,3065 s.

Apesar do aumento do custo computacional, o paralelismo continuou apresentando bom comportamento. O speedup atingiu **9,972×**, enquanto o fator de balanceamento permaneceu em **0,9902**, ou aproximadamente 99,02%.

A Figura 5 apresenta a redução do tempo de execução no cenário de estresse.

Figura 5: Tempo de Execução vs. Threads (Vale dos Cavalos-Marinhos, MAX_ITER=5000)

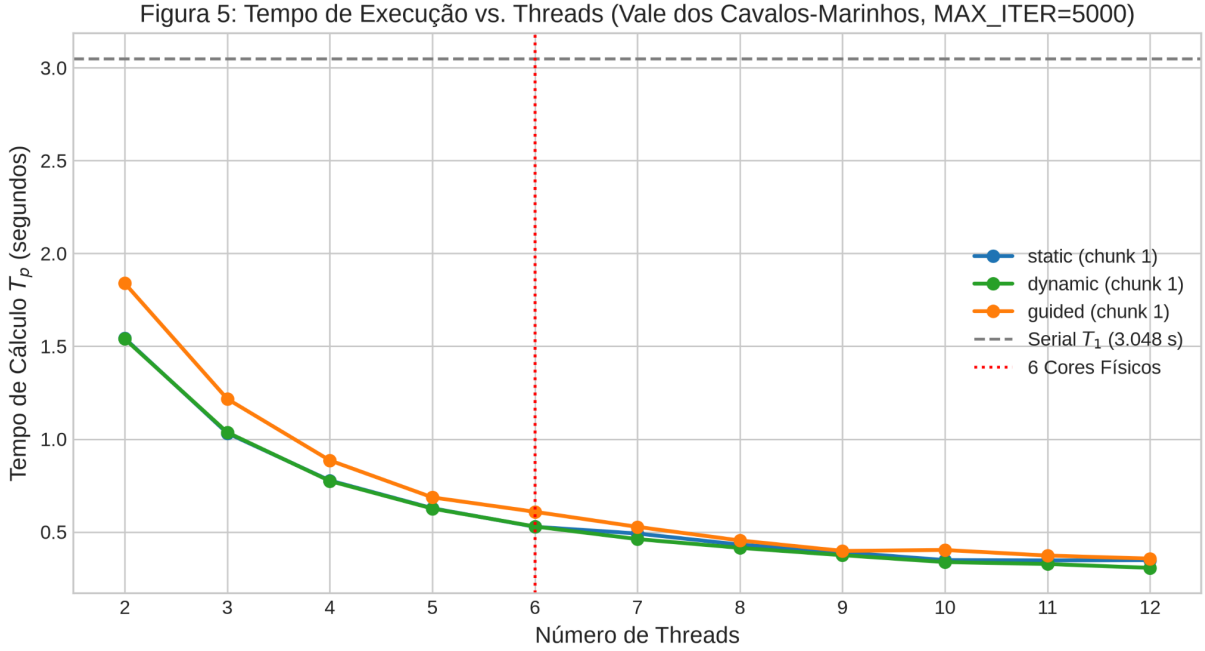


Figura 5: Tempo de execução em função do número de threads no Vale dos Cavalos-Marinhos.

A curva apresenta comportamento semelhante ao caso padrão, mas com tempos absolutos maiores. O ganho de desempenho continua significativo mesmo com o aumento de `MAX_ITER` para 5000.

3.5. Comparação do Balanceamento de Carga

O fator de balanceamento permite analisar uma característica que não pode ser observada apenas pelo tempo total: o quanto o trabalho foi distribuído de maneira uniforme entre as threads.

A Figura 6 apresenta o fator de balanceamento para diferentes números de threads, comparando as políticas e os tamanhos de chunk nos dois cenários.

Figura 6: Fator de Balanceamento de Carga (T_{avg}/T_{max}) por Política e Chunk

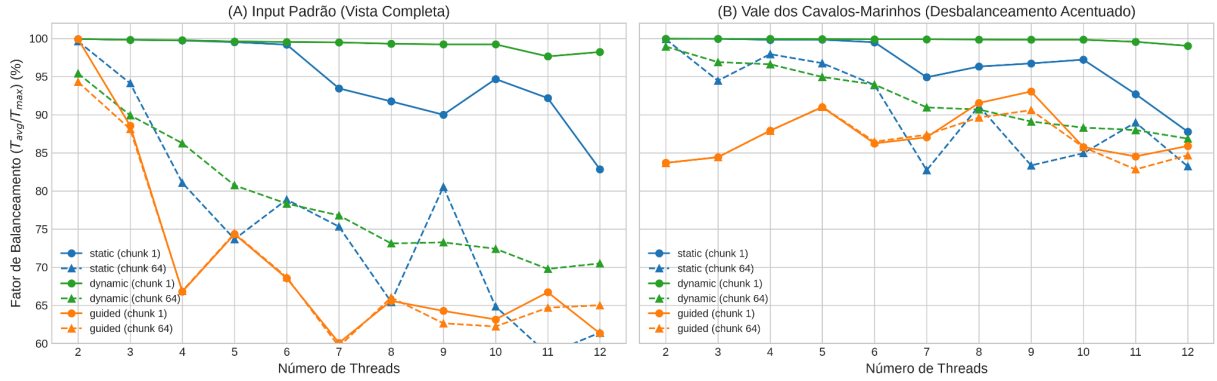


Figura 6: Fator de balanceamento de carga para diferentes políticas e tamanhos de chunk.

No input padrão, a configuração `dynamic, chunk=1` apresenta fator de balanceamento próximo de 100% em praticamente toda a faixa de threads analisada. Em contrapartida, as configurações com chunks maiores apresentam uma redução mais acentuada.

Esse comportamento é particularmente visível para `static, chunk=64`, cuja eficiência de distribuição cai à medida que aumenta o número de threads. Com 12 threads, essa configuração apresenta LBF de aproximadamente 61,4%.

No cenário do Vale dos Cavalos-Marinhos, o comportamento é diferente. Mesmo as configurações com chunks maiores apresentam valores de balanceamento relativamente elevados em alguns pontos. Isso ocorre porque a região analisada possui uma distribuição espacial do custo diferente da região completa do conjunto.

Ainda assim, `dynamic, chunk=1` mantém comportamento bastante estável, chegando a aproximadamente 99% de balanceamento com 12 threads.

Portanto, os resultados mostram que o tamanho do chunk possui influência direta na capacidade de adaptação do escalonador à heterogeneidade do custo das linhas.

3.6 Comparação do Speedup entre os cenários

A Figura 7 permite comparar diretamente o speedup obtido no input padrão e no Vale dos Cavalos-Marinhos.

Figura 7: Comparativo de Speedup: Padrão vs. Desbalanceamento Acentuado

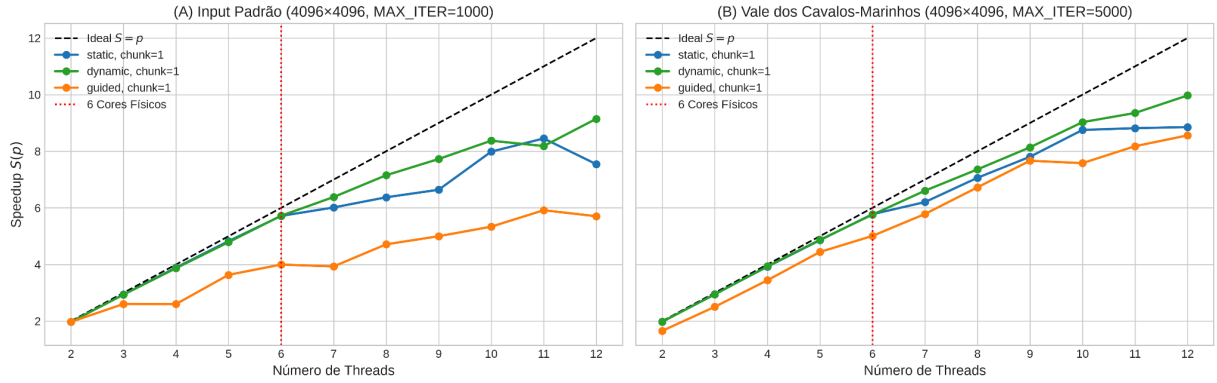


Figura 7: Comparação do speedup entre o input padrão e o Vale dos Cavalos-Marinhos.

Em ambos os cenários, o speedup cresce à medida que o número de threads aumenta. Entretanto, o comportamento das políticas de escalonamento apresenta diferenças.

No caso padrão, `dynamic, chunk=1` alcança aproximadamente 9,15× com 12 threads. No Vale dos Cavalos-Marinhos, a mesma configuração alcança aproximadamente 9,97×.

3.7 Análise de Escalabilidade Fraca e Limitação do Hardware

Também foi realizada uma análise de escalabilidade fraca (*weak scaling*), na qual o tamanho do problema é aumentado juntamente com a quantidade de recursos computacionais.

Foram considerados três pontos experimentais:

- 4096×4096 pixels com 2 threads;

- 8192×8192 pixels com 8 threads;
- 16384×16384 pixels com 12 threads.

A Figura 8 apresenta os resultados.

Figura 9: Análise de Escalabilidade Fraca (Lei de Gustafson)

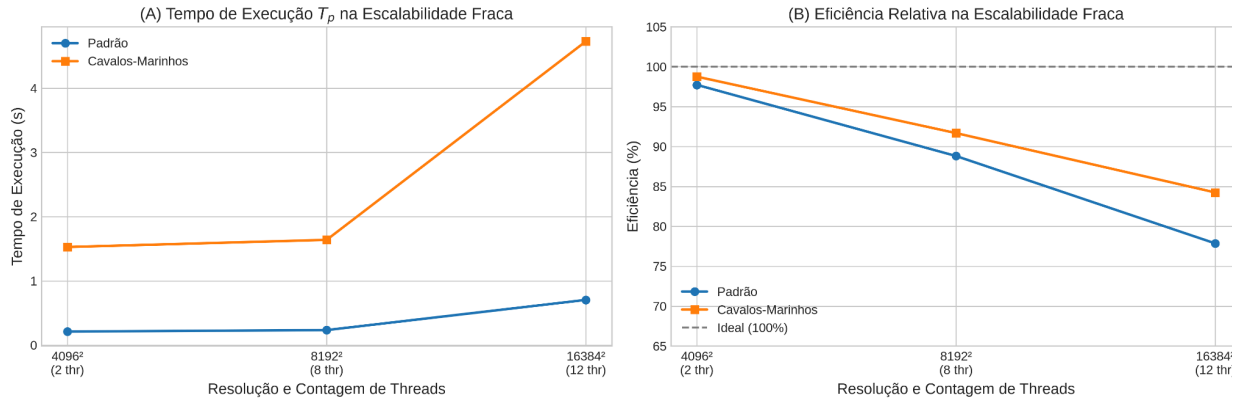


Figura 8: Análise da escalabilidade fraca para os dois cenários.

No cenário padrão, o tempo paralelo aumenta de aproximadamente 0,2107 s para 0,2338 s quando a resolução passa de 4096² para 8192² pixels, apesar do aumento correspondente do número de threads de 2 para 8.

Isso indica que a utilização de recursos adicionais consegue absorver grande parte do aumento da carga de trabalho. Entretanto, no terceiro ponto, com 16384² pixels e 12 threads, o tempo sobe para aproximadamente 0,7041 s.

No cenário do Vale dos Cavalos-Marinhos, observa-se comportamento semelhante, porém com tempos maiores devido ao custo computacional mais elevado. O tempo paralelo passa de 1,5274 s para 1,6387 s e posteriormente para 4,7300 s.

A eficiência relativa também apresenta redução progressiva. No cenário padrão, ela passa de 97,7% para 88,8% e posteriormente para 77,9%. No cenário Seahorse, os valores correspondentes são 98,8%, 91,7% e 84,2%.

Essa redução mostra que aumentar simultaneamente a carga e o número de threads não produz crescimento perfeitamente proporcional ao desempenho. Além da capacidade de processamento, entram em cena fatores como compartilhamento de recursos do processador, hierarquia de memória e o limite de 12 threads disponíveis no hardware utilizado.

É importante destacar que o terceiro ponto não mantém uma relação perfeitamente proporcional entre carga e recursos, pois a resolução aumenta em 16 vezes em relação ao primeiro cenário, enquanto o número de threads aumenta apenas de 2 para 12. A limitação decorre do número máximo de threads lógicas disponíveis na máquina.

4. Discussão

4.1 Análise da corretude da implementação

Os testes de corretude demonstraram que a versão OpenMP reproduz exatamente os resultados da implementação sequencial. Nos dois cenários analisados, os arquivos de saída apresentaram hashes SHA-256 idênticos e nenhuma divergência de pixels.

Esse resultado confirma que a paralelização preservou a lógica do algoritmo. A divisão das linhas entre as threads não introduziu alterações nos valores calculados, e as otimizações utilizadas, como a simetria axial no caso padrão, foram mantidas de forma consistente entre as duas versões. Como cada linha é processada independentemente e não há operações de redução ou dependências entre pixels, a distribuição das linhas entre threads não altera a sequência de operações realizada para cada pixel.

A corretude é particularmente importante neste experimento porque o objetivo não é produzir uma aproximação diferente do conjunto de Mandelbrot, mas executar o mesmo cálculo de maneira mais eficiente.

4.2 Comparação entre a versão sequencial e a versão paralela em OpenMP

A versão sequencial foi utilizada como referência para comparar o desempenho da versão paralela. No caso padrão, o tempo médio da execução sequencial foi de 0,420650 s.

Na versão paralela, o tempo de execução diminuiu conforme o número de threads aumentou. Com 12 threads, por exemplo, a configuração dynamic com chunk=1 apresentou tempo médio de 0,046265 s, enquanto a execução sequencial levou 0,420650 s.

Esse resultado mostra que o uso de várias threads permitiu executar o cálculo em menos tempo. Entretanto, a redução do tempo não foi proporcional ao aumento do número de threads, pois existem custos relacionados à paralelização e à divisão do trabalho.

4.3 Impacto do número de threads no desempenho

Os resultados demonstram uma relação aproximadamente linear entre o aumento do número de threads e a redução do tempo de execução até a região correspondente aos seis núcleos físicos.

No input padrão, o speedup aumenta de 1,973× com duas threads para 5,722× com seis threads. Isso corresponde a uma eficiência de 95,37% com seis threads.

No cenário Seahorse, o comportamento é semelhante, com speedup de 5,776× e eficiência de 96,27% utilizando seis threads.

A partir de sete threads, entretanto, ocorre uma redução mais acentuada da eficiência. Isso coincide com a transição entre a utilização exclusiva dos seis núcleos físicos e a utilização das threads lógicas disponibilizadas pelo SMT.

Isso não significa que as threads adicionais deixem de proporcionar benefício. Pelo contrário, o tempo continua diminuindo. Entretanto, cada thread adicional produz um ganho relativo menor porque passa a compartilhar recursos físicos de processamento com outra thread.

4.4 Desbalanceamento de Carga

O Conjunto de Mandelbrot constitui um exemplo particularmente adequado para estudar balanceamento de carga porque o número de iterações necessárias para determinar cada pixel não é constante.

Um pixel localizado em uma região externa pode escapar após poucas iterações. Em contrapartida, pixels próximos da fronteira do conjunto podem exigir centenas ou milhares de iterações.

Como o programa distribui linhas inteiras entre as threads, a heterogeneidade espacial do fractal se transforma diretamente em heterogeneidade entre as tarefas.

Uma distribuição estática com chunks grandes pode fazer com que uma thread receba várias linhas computacionalmente pesadas, enquanto outra receba linhas predominantemente leves. Nesse caso, a primeira thread continua executando enquanto as demais permanecem ociosas.

Esse fenômeno é observado nos resultados de `static, chunk=64`, principalmente no input padrão, onde o LBF chega a aproximadamente 0,6142 com 12 threads.

4.5 Comportamento das Políticas static, dynamic e guided

Os experimentos mostram diferenças claras entre as políticas de escalonamento.

A política `static` possui baixo custo de gerenciamento porque a distribuição é determinada antes da execução. Essa característica é vantajosa quando as tarefas possuem custos semelhantes. No Mandelbrot, entretanto, a irregularidade espacial pode reduzir seu desempenho quando são utilizados chunks grandes.

A política `dynamic` apresenta maior capacidade de adaptação. Quando uma thread termina uma tarefa, ela pode receber outra, permitindo redistribuir o trabalho restante de acordo com o ritmo real de execução das threads.

Essa característica explica o comportamento observado para `dynamic, chunk=1`, que apresentou os melhores resultados de tempo e balanceamento em 12 threads nos dois cenários.

A política `guided` procura reduzir o custo de gerenciamento iniciando com blocos maiores e diminuindo progressivamente seu tamanho. Embora essa estratégia possa ser eficiente em determinados problemas, nos experimentos realizados ela não superou `dynamic, chunk=1`.

Portanto, os resultados experimentais indicam que a irregularidade do custo das linhas do Mandelbrot favoreceu uma política capaz de redistribuir frequentemente o trabalho.

4.6 Efeito do Tamanho do Chunk

O tamanho do chunk apresentou influência significativa sobre o desempenho.

Com `chunk=1`, cada tarefa corresponde a uma única linha da imagem. Isso permite que as threads alternem rapidamente entre regiões com diferentes custos computacionais.

Com `chunk=16`, cada atribuição representa 16 linhas consecutivas. Já com `chunk=64`, uma única atribuição representa 64 linhas.

No caso padrão, o aumento da granularidade de 1 para 64 linhas provocou uma redução significativa do speedup com 12 threads: de 9,147× para aproximadamente 6,544× no `dynamic` e 6,050× no `static`.

A Figura 6 evidencia que essa redução de desempenho está acompanhada por uma queda do fator de balanceamento. Portanto, os resultados sugerem que o agrupamento

de linhas com custos heterogêneos exerce papel importante na redução observada do balanceamento.

4.7 Input padrão versus Vale dos Cavalos-Marinhos

A comparação entre os dois cenários permite observar duas situações diferentes do mesmo algoritmo.

O input padrão representa uma região ampla do conjunto, utilizando `MAX_ITER=1000`. O Vale dos Cavalos-Marinhos utiliza uma região muito menor, mas com `MAX_ITER=5000`, produzindo um cenário computacionalmente mais pesado.

Apesar da diferença de custo absoluto, a implementação paralela apresentou comportamento consistente nos dois casos.

Com 12 threads e `dynamic, chunk=1`, o speedup foi de aproximadamente 9,15× no padrão e 9,97× no Seahorse. O LBF também permaneceu elevado, chegando a 98,22% e 99,02%, respectivamente.

Isso indica que o aumento da carga computacional por tarefa não impediu que a distribuição dinâmica mantivesse elevado fator de balanceamento nos testes realizados. Pelo contrário, a maior quantidade de trabalho por linha também reduz a importância relativa do custo associado ao gerenciamento das tarefas.

4.8 Escalabilidade, Gargalos e Limitações

A análise de escalabilidade fraca mostrou que o sistema consegue absorver parte significativa do aumento da carga quando mais threads são utilizadas. No cenário padrão, o tempo paralelo passou de aproximadamente 0,2107 s, com 4096×4096 pixels e 2 threads, para 0,2338 s com 8192×8192 pixels e 8 threads. No terceiro ponto, com 16384×16384 pixels e 12 threads, o tempo aumentou para aproximadamente 0,7041 s. No cenário do Vale dos Cavalos-Marinhos, os tempos correspondentes foram de 1,5274 s, 1,6387 s e 4,7300 s.

A redução da eficiência observada nos pontos de maior carga indica que o aumento da quantidade de recursos não compensa de forma perfeitamente proporcional o crescimento do problema. No cenário padrão, a eficiência passou de 97,7% para 88,8% e posteriormente para 77,9%. No cenário do Seahorse, os valores foram de 98,8%, 91,7% e 84,2%. Parte desse comportamento está relacionada às características do hardware utilizado, que possui seis núcleos físicos e 12 threads lógicas. Assim, a avaliação ficou limitada a, no máximo, 12 threads, não sendo possível verificar o comportamento da aplicação em arquiteturas com maior número de núcleos.

Outro fator observado nos experimentos foi o impacto do balanceamento de carga. Como o custo computacional das linhas não é uniforme, determinadas configurações de escalonamento e chunk podem deixar algumas threads com mais trabalho do que outras. Esse comportamento foi particularmente evidente no caso padrão com `static, chunk=64`, que apresentou LBF de aproximadamente 0,6142 com 12 threads, enquanto `dynamic, chunk=1` atingiu 0,9822. Dessa forma, a distribuição das tarefas constitui um dos principais fatores que limitam o aproveitamento dos recursos disponíveis.

O tamanho da carga também impõe limitações à avaliação. Na resolução de 16384×16384 pixels, são processados aproximadamente 268 milhões de pixels, aumentando significativamente a quantidade de dados produzidos e a demanda sobre a hierarquia de memória. Entretanto, os experimentos realizados não permitem determinar isoladamente quanto desse comportamento é causado pela computação, pela memória ou pelo gerenciamento das threads, uma vez que esses fatores não foram medidos separadamente.

Outra limitação está relacionada ao escopo da medição. O tempo utilizado para calcular o desempenho considera apenas o processamento dos pixels, não incluindo a gravação dos arquivos `.bin` e `.ppm`. Portanto, os resultados apresentados representam o custo da etapa de cálculo e não o tempo total de execução do programa.

Por fim, os resultados foram obtidos em uma única configuração de hardware e software. Consequentemente, os valores de tempo, speedup e eficiência não devem ser generalizados diretamente para outras arquiteturas. Diferentes quantidades de núcleos, capacidades de cache, mecanismos de SMT e características do runtime OpenMP podem produzir comportamentos distintos.

4.9 Possíveis melhorias

A partir dos resultados obtidos, podem ser consideradas algumas melhorias para ampliar a análise de desempenho e explorar outras possibilidades de otimização da implementação. Uma primeira possibilidade seria avaliar diferentes configurações de afinidade e posicionamento das threads, especialmente considerando que o processador utilizado possui seis núcleos físicos e 12 threads lógicas. Testes com diferentes políticas de associação entre threads e núcleos poderiam permitir uma análise mais detalhada do impacto do SMT sobre o desempenho.

Outra possibilidade seria investigar diferentes tamanhos de *chunk* além dos valores utilizados neste trabalho. Os experimentos mostraram que o tamanho da unidade de trabalho influencia tanto o tempo de execução quanto o balanceamento de carga. Dessa forma, testar valores intermediários entre 1, 16 e 64 poderia ajudar a identificar

configurações que apresentem uma relação adequada entre granularidade das tarefas e custo de gerenciamento do escalonamento.

Outra melhoria seria ampliar os experimentos para diferentes resoluções e valores de `MAX_ITER`. O trabalho avaliou principalmente os cenários de 4096 × 4096 pixels com `MAX_ITER = 1000` e do Vale dos Cavalos-Marinhos com `MAX_ITER = 5000`. A utilização de outros tamanhos permitiria verificar se o comportamento observado para as políticas de escalonamento e tamanhos de *chunk* permanece consistente em cargas menores e maiores.

De modo geral, essas melhorias permitiriam ampliar a avaliação realizada nesta etapa, investigando não apenas a configuração de escalonamento que apresentou os melhores resultados nos experimentos atuais, mas também a influência da arquitetura, da granularidade das tarefas sobre o desempenho do algoritmo.

5. Conclusão

5.1 Principais aprendizados da Etapa 1

A realização desta etapa permitiu entender, na prática, como uma aplicação sequencial pode ser paralelizada utilizando o OpenMP. Foi possível observar como o trabalho pode ser dividido entre diferentes threads e como essa divisão pode reduzir o tempo de execução em relação à versão sequencial.

Também foi possível entender a importância de escolher uma política de escalonamento adequada e um tamanho de `chunk` que favoreça a distribuição do trabalho. Os testes mostraram que o desempenho não depende apenas da quantidade de threads, mas também da forma como o trabalho é distribuído entre elas.

Outro aprendizado importante foi a necessidade de verificar a corretude da versão paralela. Antes de analisar o desempenho, foi necessário garantir que os resultados produzidos pelo OpenMP fossem iguais aos da versão sequencial.

5.2 Considerações sobre o desempenho e o balanceamento de carga

Os resultados mostraram que a utilização de memória compartilhada com OpenMP conseguiu reduzir o tempo de execução do cálculo do conjunto de Mandelbrot. No caso padrão, o melhor resultado com 12 threads foi obtido com `dynamic` e `chunk=1`, com tempo médio de 0,046265 s, speedup médio de 9,147 e eficiência média de 76,22%.

Entretanto, o aumento do número de threads não trouxe ganhos proporcionais em todos os casos. A eficiência diminuiu com o aumento do número de threads, mostrando que existem custos relacionados à paralelização e à divisão do trabalho.

O balanceamento de carga também foi um ponto importante. Como diferentes pixels podem precisar de diferentes quantidades de iterações, algumas partes da imagem possuem um custo maior de processamento. No caso do Seahorse, o uso de diferentes políticas de escalonamento permitiu analisar como a distribuição do trabalho afeta o balanceamento. A configuração `dynamic` com `chunk=1` apresentou um bom balanceamento, com LBF de 0,9902 utilizando 12 threads.

Assim, os testes mostraram que, além de aumentar o número de threads, é necessário escolher uma forma adequada de distribuir o trabalho para aproveitar melhor os recursos disponíveis.

5.3 Comparação com a paralelização do DGEMM

O DGEMM estudado em sala de aula realiza a multiplicação de matrizes. No código utilizado na disciplina, essa operação é realizada por meio de laços que percorrem as linhas, colunas e elementos das matrizes. O programa também realiza várias repetições e mede o tempo de execução e o desempenho para diferentes tamanhos de matrizes.

O Mandelbrot possui uma característica diferente. Apesar de também permitir que o trabalho seja dividido entre diferentes threads, o custo para calcular cada pixel não é sempre igual. Alguns pixels precisam de poucas iterações, enquanto outros podem precisar de muitas iterações. Dessa forma, dependendo de como as linhas são distribuídas, algumas threads podem receber mais trabalho do que outras, causando desbalanceamento da carga.

Essa diferença foi importante nos testes realizados neste projeto. No Mandelbrot, foram analisadas diferentes políticas de escalonamento e tamanhos de `chunk` para encontrar uma distribuição de trabalho mais adequada. A configuração `dynamic` permite que novas partes do trabalho sejam distribuídas conforme as threads terminam suas tarefas. Nos testes realizados, `dynamic` com `chunk=1` apresentou bons resultados de tempo e balanceamento.

Assim, a principal diferença entre os dois casos é que a multiplicação de matrizes do DGEMM possui um trabalho mais regular, enquanto o Mandelbrot apresenta uma carga de trabalho mais irregular. No Mandelbrot, essa irregularidade torna o balanceamento entre as threads um aspecto importante para o desempenho. Os dois exemplos

mostram, portanto, a importância de uma boa divisão do trabalho em aplicações paralelas.

6. Referências

ETVORELLANA. **meuGEMM**. GitHub. Disponível em: https://github.com/etvorellana/meuGEMM. Acesso em: 21 set. 2026.

ETVORELLANA. **RepoDEC107-PP-2026_2**. GitHub. Disponível em: https://github.com/etvorellana/RepoDEC107-PP-2026_2. Acesso em: 21 set. 2026.

OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP API 6.0 Reference Guide*. 2024. Disponível em: https://www.openmp.org/wp-content/uploads/OpenMP-RefGuide-6.0-OMP60SC24-web.pdf. Acesso em: 21 set. 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Enzo Fernandes Sarmento e Jéssica Meira Costa, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: Codex, Antigravity CLI, Gemini. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Revisão gramatical do relatório
- Geração de scripts base para testes de benchmark
- Geração de scripts python para plotagem dos gráficos de acordos com os dados extraídos
- Estruturação inicial de ideias e revisão bibliográfica

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 22 de setembro de 2026

Enzo Fernandes Sarmento - Matrícula: 202410024
Jéssica Meira Costa - Matrícula: